

ZEBRA RFID AI PROJECT NOTE

Zebra RFID AI Application & SDK Sample

Android RFID Application & Gemini AI Developer Assistant

AUTHOR	Chuck Lin
DATE	September 29, 2026
REFERENCE	ZEBRA-RFID-AI

Project architecture, setup requirements, build instructions, and Gemini AI code assistant notes for the Zebra RFID Android application.

Contents

Zebra RFID AI - Technical Design Document & Architecture **2**

1. System Architecture & Connection State Flow 2

2. Sequence Diagram: Connection & Lifecycle Flow 2

3. UI Screenshots & Visual Interface 3

4. Code Implementation Highlights 3

Zebra RFID AI - Technical Design Document & Architecture

This document outlines the architecture, connection flows, activity lifecycle support (suspend & resume), inventory operations, and **Gemini 3.8 AI** assistant integration for the **Zebra RFID AI** Android application.

1. System Architecture & Connection State Flow

All Zebra RFID SDK operations are serialized on a single-threaded background executor (**ExecutorService**) in **RfidHandler.kt** to guarantee thread safety and maintain a smooth **60fps Jetpack Compose UI**.

```
%%{init: {"theme":"base","themeVariables":{"primaryColor":"#E8F3F0","primaryBorderColor":"#1F7A6B"}
flowchart TB
    A[App visible] --> B[DISCOVERING: create Readers, scan transports]
    B --> C{Readers found?}
    C -- None --> F[NO_READERS: dispose SDK]
    C -- One --> E[CONNECTING]
    C -- Several --> P{Saved reader present?}
    P -- Yes --> E
    P -- No --> S[Reader picker]
    S -- Picked --> E
    E --> D{Connected?}
    D -- Yes --> G[CONNECTED: ready for inventory]
    D -- No --> X[FAILED] --> S
    G --> H[App background]
    H --> I[DISCONNECTED: dispose SDK]
    I --> A
    F -- Connect pressed --> B
    J[Reader appears] --> E
    K[Reader disappears] --> L[DISCONNECTED]

    classDef core fill:#E8F3F0,stroke:#1F7A6B,color:#142B3A
    classDef decision fill:#F7F1DF,stroke:#B9852F,color:#142B3A
    classDef event fill:#F1F4F5,stroke:#5C6B73,color:#142B3A
    class C,P,D decision
    class J,K event
    class A,B,E,G,H,I,L core
```

2. Sequence Diagram: Connection & Lifecycle Flow

```
%%{init: {"theme":"base","themeVariables":{"primaryColor":"#E8F3F0","primaryBorderColor":"#1F7A6B"}
sequenceDiagram
    participant App as MainActivity
    participant Handler as RFIDHandler (executor)
```

```

participant SDK as Zebra RFID SDK
participant Reader as ReaderDevice/RFIDReader

App->>Handler: onForeground(this, this)
Handler-->>App: DISCOVERING
Handler->>SDK: new Readers(appContext, transport)
Handler->>SDK: Readers.attach(this)
Handler->>SDK: scanTransports()
SDK-->>Handler: available readers list
Handler->>Handler: getAvailableReader() / saved reader
Handler-->>App: CONNECTING
Handler->>Reader: reader.connect()
Handler->>Reader: enableEvents(eventHandler)
Handler-->>App: CONNECTED

App->>Handler: onBackground()
Handler->>Reader: removeEventListener() + disconnect()
Handler-->>App: DISCONNECTED
Handler->>SDK: Readers.detach(this)
Handler->>SDK: readers.Dispose()

App->>Handler: onForeground(this, this)
Handler-->>App: DISCOVERING
Handler->>SDK: new Readers(...)
Handler->>Reader: connect() to saved reader
Handler-->>App: CONNECTED

```

3. UI Screenshots & Visual Interface

RFID Inventory Scanner Screen

The scanner UI provides real-time connection status, unique tag counts, total reads, EPC filter search bar, and inventory start/stop controls. RFID Scanner

Gemini AI Assistant Screen

The AI assistant tab features preset prompt chips for Zebra RFID SDK code snippets and inventory analysis. Gemini AI Screen

Gemini AI Code Response

Gemini 3.8 AI generated code explanation with local caching (⚡ **Cached**) and force-refresh support. Gemini Response

4. Code Implementation Highlights

A. RFID Initialization, Discovery & Region Setup (RfidUtility.kt)

```
object RfidUtility {
    const val DEFAULT_READER_NAME = "RFID-TC701"

    @Throws(InvalidUsageException::class)
    fun scanTransports(readers: Readers): ArrayList<ReaderDevice> {
        for (candidate in ENUM_TRANSPORT.values()) {
            if (candidate == ENUM_TRANSPORT.ALL) continue
            try {
                readers.setTransport(candidate)
                val found = readers.GetAvailableRFIDReaderList()
                if (found != null && found.isNotEmpty()) return found
            } catch (_: InvalidUsageException) {}
        }
        return ArrayList()
    }

    fun configureRegion(reader: RFIDReader, regionCode: String = "USA"): String? {
        try {
            val regCfg = reader.Config.regulatoryConfig
            val supportedRegions = reader.ReaderCapabilities.SupportedRegions
            for (i in 0 until supportedRegions.length()) {
                val regionInfo = supportedRegions.getRegionInfo(i)
                if (regionCode.equals(regionInfo.regionCode, ignoreCase = true)) {
                    regCfg.region = regionInfo.regionCode
                    regCfg.setIsHoppingOn(regionInfo.isHoppingConfigurable)
                    regCfg.setEnabledChannels(regionInfo.supportedChannels)
                    regCfg.setStandardName(regionInfo.name)
                    reader.Config.regulatoryConfig = regCfg
                    return null
                }
            }
            return "Region $regionCode not supported"
        } catch (e: Exception) {
            return e.message
        }
    }
}
```

B. Gemini 3.8 AI Assistant with Local Caching (RfidAiViewModel.kt)

```
class RfidAiViewModel(application: Application) : AndroidViewModel(application) {
    private val generativeModel = Firebase.ai.generativeModel("gemini-3.8-flash")
    private val sharedPreferences = application.getSharedPreferences("gemini_cache", Context.MODE_PRIVATE)
    private val responseCache = ConcurrentHashMap<String, String>()

    fun askQuestion(userPrompt: String, scannedTags: List<TagItem> = emptyList(), forceRefresh: Boolean = false) {
        val cacheKey = userPrompt.trim().lowercase()
        if (!forceRefresh && responseCache.containsKey(cacheKey)) {
            _uiState.value = RfidAiUiState.Success(responseCache[cacheKey]!!, isCached = true)
            return
        }
    }
}
```

```
viewModelScope.launch(Dispatchers.IO) {  
    val response = generativeModel.generateContent(content { text(userPrompt) })  
    response.text?.let { output ->  
        responseCache[cacheKey] = output  
        sharedPreferences.edit().putString(cacheKey, output).apply()  
        _uiState.value = RfidAiUiState.Success(output, isCached = false)  
    }  
}  
}
```